

SQL Query Rules Cheat Sheet

1. Logical SQL Execution Order

Although we write:

SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT

SQL processes them approximately as:

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

This explains many alias-related errors.

2. Column Aliases Cannot Be Used in WHERE

✗ Invalid

```
SELECT salary AS sal
FROM Employee
WHERE sal > 5000;
```

Reason: `WHERE` executes before `SELECT`, so `sal` does not exist yet.

✓ Correct

```
SELECT salary AS sal
FROM Employee
WHERE salary > 5000;
```

3. Column Aliases Can Usually Be Used in ORDER BY

✓ Valid

```
SELECT salary AS sal
FROM Employee
ORDER BY sal;
```

Reason: `ORDER BY` executes after `SELECT` .

4. Every FROM Must Have a Table or Subquery

✗ Invalid

```
SELECT *  
FROM  
WHERE age > 20;
```

✓ Correct

```
SELECT *  
FROM Person  
WHERE age > 20;
```

5. Every Derived Table (Subquery in FROM) Needs an Alias

✗ Invalid

```
SELECT *  
FROM (  
    SELECT *  
    FROM Employee  
);
```

✓ Correct

```
SELECT *  
FROM (  
    SELECT *  
    FROM Employee  
) AS e;
```

6. Table Aliases Are Available in WHERE

✓ Valid

```
SELECT *  
FROM Employee e  
WHERE e.salary > 5000;
```

Reason: Table aliases are created during the FROM phase.

7. Inner Query Column Aliases Can Be Used in the Outer Query

✓ Valid

```
SELECT *  
FROM (  
    SELECT salary * 12 AS annual_salary  
    FROM Employee  
) p  
WHERE p.annual_salary > 100000;
```

Reason: The subquery result behaves like a temporary table.

8. Aggregate Functions Reduce Rows

Common aggregate functions:

- SUM()
- COUNT()
- AVG()
- MIN()
- MAX()

Example:

```
SELECT SUM(weight)  
FROM Queue;
```

Returns one row.

9. WHERE vs HAVING

WHERE

Filters rows before grouping.

```
SELECT *  
FROM Employee  
WHERE salary > 5000;
```

HAVING

Filters groups after grouping.

```
SELECT department_id  
FROM Employee  
GROUP BY department_id  
HAVING COUNT(*) > 5;
```

Rule:

- Row-level condition → WHERE
- Aggregate condition → HAVING

10. GROUP BY Rule

When using GROUP BY:

- Every selected column must either:
 - Appear in GROUP BY, or
 - Be aggregated.

✗ Invalid

```
SELECT department_id, employee_name  
FROM Employee  
GROUP BY department_id;
```

✓ Valid

```
SELECT department_id, COUNT(*)  
FROM Employee  
GROUP BY department_id;
```

11. IN Operator Rule

✓ Valid

```
WHERE id IN (1, 2, 3)
```

✓ Valid

```
WHERE id IN (  
    SELECT id  
    FROM Orders  
)
```

✗ Invalid

```
WHERE salary = 5000 IN (...)
```

12. Aggregate Aliases and WHERE

✗ Invalid

```
SELECT COUNT(*) AS total  
FROM Employee  
WHERE total > 10;
```

✓ Correct

```
SELECT COUNT(*) AS total  
FROM Employee  
HAVING total > 10;
```

13. Window Functions Do Not Reduce Rows

Aggregate:

```
SELECT SUM(weight)
FROM Queue;
```

Returns one row.

Window Function:

```
SELECT
    person_name,
    SUM(weight) OVER (ORDER BY turn) AS running_total
FROM Queue;
```

Returns every row plus the running total.

14. Correlated Subqueries Can Reference Outer Query Columns

✓ Valid

```
SELECT *
FROM Employee e
WHERE salary >
(
    SELECT AVG(salary)
    FROM Employee
    WHERE department_id = e.department_id
);
```

The inner query can reference columns from the outer query.

15. SQL Debugging Checklist

Whenever a query fails, ask:

1. Did I specify a table after FROM?
2. Did I define every alias before using it?
3. Am I using a SELECT alias inside WHERE?

4. Does every subquery in FROM have an alias?
 5. Should I use WHERE or HAVING?
 6. Am I mixing aggregated and non-aggregated columns?
 7. Is GROUP BY required?
 8. Does my subquery return one row or multiple rows?
 9. Should I use IN, EXISTS, or = ?
 10. Would a window function simplify the problem?
-

Golden Rule

Same Query Block:

- SELECT alias → ❌ Not available in WHERE
- SELECT alias → ✅ Available in ORDER BY

Outer Query:

- Column alias from an inner subquery → ✅ Available everywhere in the outer query

Think of a subquery as a temporary table. Any aliases created inside it become actual column names for the outer query.